今天学习总结：

1.codex一些插件的使用和基础知识

2.申请机器并阅读三个仓库的 README 和目录结构

3.整理整个技术链路

1.codex学习

注册账号我在先前就注册过了，主要是看视频差缺补漏有没有什么没学的。比如上下文，命令行，图片生成等等。

两大记忆系统：手动记忆：agents.md

自动记忆：把以前的对话进行记录。自动记忆规则：过短不记录，没额度不记录。

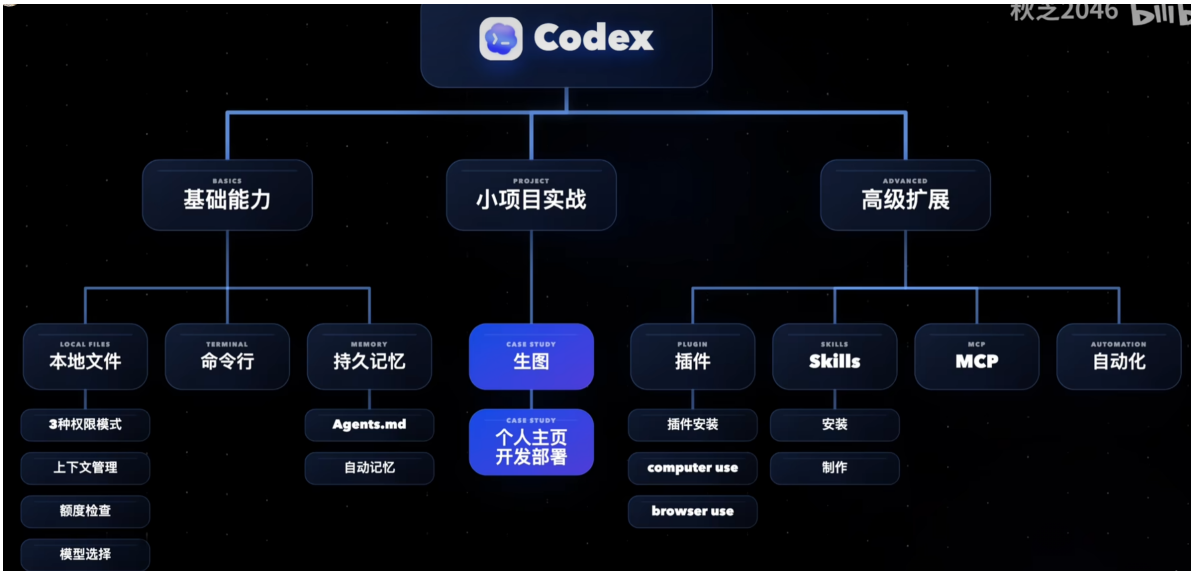
agents.md分为全局记忆：适用于任何规则 项目级别的md：只在项目中生效。

上下文压缩：文章过长会自动触发，也可以自己进行压缩。

MCP：接外部工具

学到了一些新的功能：可以追加，而不打断对话。多安装了一些可能能用到的插件。

codex使用端全功能如下：



我自己补充了一些Agent相关知识：

1.1Agent loop：输入 → 模型 → tool call → 本地执行 → 结果回填 → 再循环

Agent**终止条件有三层**

（1）模型不再调工具（最弱）

（2）harness 的轮数上限（兜底）

（3）**外部验收失败就停**（最重要，也最容易被忽略）

1.2Harness 管什么

事	具体
上下文压缩	阈值触发，保留摘要丢细节
轮数上限	防死循环
权限	哪些动作需要审批
沙箱	文件 / 网络 / 设备访问边界

1.3 三种常见失败模式

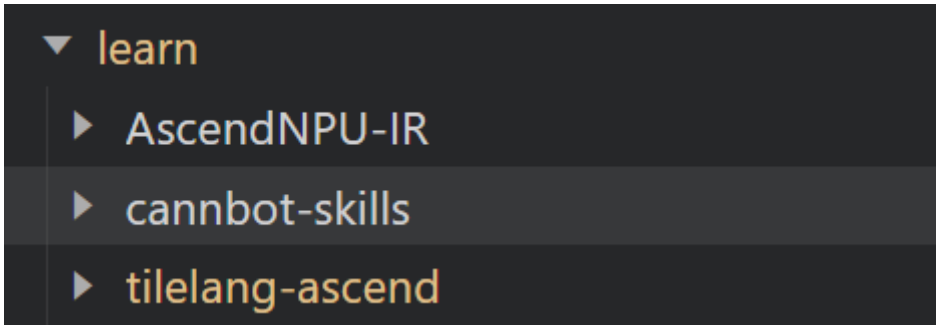
失败模式	现象	根因
反复打转	同一个错误改 5 次还是错	错误原文被压缩掉了 / 没有 ground truth
猜 API	编出 `T.xxx` 这种不存在的接口	`AGENTS.md` 没告诉它去哪查
长任务跑偏	跑了 2 小时不知道在干什么	状态只存在上下文里，被压掉了

2.申请机器

申请了一个这样的机器。跑了一下基础的。

```
developer@a6c98e6836c04614baed206dda1faa87:/mnt/workspace$ python3 -c "import torch, torch_npu
; print('device:', torch.npu.get_device_name(0)); print('available:',torch.npu.is_available())
; a=torch.randn(512,512,device='npu'); print('matmul:', (a@a).sum().item())"
/opt/buildtools/Python-3.11.4/lib/python3.11/site-packages/torch_npu/utils/_path_manager.py:66
: UserWarning: Permission mismatch: The owner of /opt/buildtools/Python-3.11.4/lib/python3.11/
site-packages/torch_npu/lib/libop_plugin_atb.so does not match.
  warnings.warn(f"Permission mismatch: The owner of {path} does not match.")
device: Ascend910B3
available: True
matmul: 21914.71875
```

同时把需要学习的仓库clone了。



阅读了一下三个项目的md:

AscendNPU-IR:昇腾的 MLIR 方言集合和编译工具链，是 CANN 栈里的编译器层。

它是"编译昇腾算子的中间表示"。给了两层接口，一是高层封装（把计算、数据搬运、同步指令包成好用的抽象），二是细粒度控制（片上内存地址、同步插入点、ping-pong 流水开关）。外加编译时自动识别硬件架构，把硬件无关的表达映射到具体指令。

源码全在 `bishengir/` 下：`include` 头文件、`lib` 源码、`test` 测试（其中 `Integration` 是端到端用例）、`tools` 是命令行工具。另外有 `build-tools` 和 `docs`。看代码从 `bishengir/lib` 入手，跑例子从 `bishengir/test/Integration` 入手。

cannbot-skills:CANNBot 智能体层的技能仓——把算子开发知识拆成 Agent 可加载的 Skill 文件，不参与编译。

CANNBot 是 CANN 社区的"智能体层"，这个仓是它的技能仓：把算子开发的领域知识拆成 Agent 能加载的 Skill 文件。三层结构 Plugin → Agent → Skill，用自然语言驱动算子开发，覆盖 AscendC、PyPTO、TileLang、Triton 几条语言路线。

代码目录按 ops / model / graph / runtime / infra / tools 分了领域，tools 下是 asys、msaicerr、npu-objdump 这些工具链技能。三层架构如下：

- Plugin：用 AGENTS.md 定义入口和协作顺序，同时管芯片路由（910/950）、阶段门禁、重试和状态文件；
- Agent：README 示意图抽象成 architect / developer / reviewer / tester 四类；实际角色随插件不同——AscendC 注册调用插件是 7 个角色，tilelang 线是analyst / developer / perf-tuner；
- Skill：知识和模板本体，分知识库（npu-arch、tiling 方法论、API 最佳实践、精度标准）、工程模板、调试测试（精度 / 运行时 / 崩溃 / 同步审计 / 环境检查）、测试开发、profiling。

tilelang-ascend:昇腾版的 TileLang

昇腾版的 TileLang。写还是写 TileLang 的 Python DSL，编译走 TVM，最后由 AscendNPU-IR 变成昇腾能跑的二进制。

定位：一是它是 tile-lang 的昇腾变体，复用 tile-lang 的 Python 写法和 TVM；二是后端有两条路，npuir 分支走 AscendNPU IR，ascendc_pto 分支走 Ascend C & PTO。

性能 benchmark。 三组数：GEMM 0.98x AscendC，DeepSeek-V4 mHC 平均 0.96x，Flash Attention 0.95x。另外 hc_sinkhorn 在大 shape（b*s=16384）反而到了 1.028x。

3.理顺链路图：

人的需求（例如"实现一个 abs 算子，精度对齐 torch"）交由 Agent 处理。Agent 以"模型—工具调用—执行—结果回填"的循环工作，每一步的做法由 cannbot Skills 按 design、develop、perf三个阶段以文件形式注入，而非由模型自行发挥。Agent 产出 TileLang Python kernel 后，TileLang 编译前端先将其降为 TVM IR，再由 codegen 转为 MLIR（TileLangIR）；该 MLIR 交由 AscendNPU-IR处理：bishengir-compile 先执行 HFusion 的硬件无关融合与图优化，再执行 HIVM 的硬件相关降级，包括 Cube/Vector 核拆分、片上内存规划与流水同步插入，最终生成 NPU 可执行的二进制。二进制由 TileLang 运行时经 torch_npu 下发到卡上执行；正确性以 torch 输出为基准、用 torch.testing.assert_close 校验，性能通过 torch_npu.profiler 采集。任一项不达标，评测结果回流给 Agent，进入下一轮迭代。

